



Великотърновски университет „Св. св. Кирил и Методий“
Факултет „Математика и информатика“



Курсов проект

ПО
учебна дисциплина

„ Шаблони за софтуерен дизайн I част “

Тема

Заведение за бързо хранене

РАЗРАБОТИЛ:

Даниел Ангелов Иванов
спец. „Софтуерно инженерство“
фак. № 2409011668

РЪКОВОДИТЕЛ:

доц. д-р Златко Георгиев Върбанов
Пламенна Викторова Петрова

*Велико Търново
2025-26 г.*

Съдържание:

1. Увод
2. Анализ на проекта
3. Използвани шаблони за проектиране
 - 3.1. Factory Pattern
 - 3.2. Decorator Pattern
 - 3.3. Strategy Pattern
 - 3.4. Singleton Pattern
 - 3.5. Builder Pattern
4. UML (описателно)
5. Още основни класове
 - 5.1. Клас Dish
 - 5.2. Клас DishFactory
 - 5.3. Клас DishDecorator
 - 5.4. Клас CheeseDecorator
 - 5.5. Клас Customer
 - 5.6. Интерфейс IDiscountStrategy
 - 5.7. Клас RegularCustomerDiscount
 - 5.8. Клас Order
 - 5.9. Клас ReceiptBuilder
 - 5.10. Клас StatisticsManager
6. Реализация на системата
7. Тестване на приложението
8. Заключение
9. Използвана литература

1. Увод

Настоящият курсов проект има за цел да реализира конзолно приложение, което симулира дейността на заведение за бързо хранене. Основната идея е да се моделират процесите по:

- създаване на меню
- обработка на клиентски поръчки
- прилагане на отстъпки
- генериране на касови бележки
- събиране на статистика

Проектът е реализиран с използване на обектно-ориентирано програмиране и основни шаблони за софтуерен дизайн, които осигуряват гъвкавост, разширяемост и лесна поддръжка.

2. Анализ на проекта

Системата включва няколко основни компонента:

- Ястия и меню
- Клиенти
- Поръчки
- Отстъпки
- Касови бележки
- Статистика

3. Използвани шаблони за проектиране

◇ 3.1 Factory Pattern

Използва се за създаване на различни видове ястия.

👉 Причина:

- Лесно добавяне на нови ястия
- Скриване на логиката за създаване

◇ 3.2 Decorator Pattern

Използва се за модифициране на ястия (добавки, премахване на съставки)

👉 Пример:

- Бургер + сирене
- Бургер без лук

◇ 3.3 Strategy Pattern

Използва се за различни видове отстъпки

👉 Пример:

- Нов клиент → 0%
- Лоялен клиент → 10%
- VIP → 20%

♦ 3.4 Singleton Pattern

Използва се за:

- Управление на системата
- Статистика

♦ 3.5 Builder Pattern

Използва се за създаване на касов бон

4. UML (описателно)

Основни класове:

- Dish (абстрактен)
- Burger, Pizza, Drink
- DishFactory
- Order
- Customer
- DiscountStrategy
- ReceiptBuilder
- StatisticsManager

5. Още основни класове

В системата са реализирани няколко основни класа, които изграждат структурата на приложението и реализират основните функционалности на заведението за бързо хранене.

5.1 Клас Dish

Класът Dish представлява абстрактен базов клас за всички видове ястия в системата. Той съдържа основна информация за всяко ястие:

- име
- цена
- описание

Този клас служи като основа за всички конкретни ястия.

Основни методи:

- GetDescription()

Основни свойства:

- Name
- Price

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Project49
{
    10 references
    public abstract class Dish
    {
        3 references
        public string Name { get; set; }
        6 references
        public virtual double Price { get; set; }
        3 references
        public abstract string GetDescription();
    }
}
```

5.2 Клас DishFactory

Класът DishFactory реализира Factory Pattern.

Неговата основна задача е:

- да създава различни видове ястия
- да централизира логиката по създаване на обекти

Предимството на този подход е, че при добавяне на ново ястие не се налагат промени в основната логика на програмата.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Project49.Dishes;

namespace Project49.Factory_Pattern
{
    2 references
    public class DishFactory
    {
        2 references
        public static Dish CreateDish(string type)
        {
            switch (type)
            {
                case "burger": return new Burger();
                default: throw new Exception("Невалиден мун");
            }
        }
    }
}
```

5.3 Клас DishDecorator

Класът DishDecorator представлява абстрактен декоратор, който позволява модификация на ястията.

Чрез него могат да се:

- добавят допълнителни съставки
- премахват съставки
- променя крайната цена

Decorator Pattern позволява динамично разширяване на функционалността без промяна на оригиналния клас.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Project49.Decorator_Pattern
{
    3 references
    public abstract class DishDecorator : Dish
    {
        protected Dish dish;

        1 reference
        public DishDecorator(Dish dish)
        {
            this.dish = dish;
        }
    }
}
```


5.4 Клас CheeseDecorator

Класът CheeseDecorator наследява DishDecorator.

Той добавя:

- сирене към ястието
- допълнителна цена
- промяна в описанието

Това позволява персонализиране на поръчките според предпочитанията на клиента.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Project49.Decorator_Pattern
{
    3 references
    public class CheeseDecorator : DishDecorator
    {
        2 references
        public CheeseDecorator(Dish dish) : base(dish) { }

        2 references
        public override string GetDescription()
        {
            return dish.GetDescription() + ", сирене";
        }

        6 references
        public override double Price => dish.Price + 1.00;
    }
}
```

5.5 Клас Customer

Класът Customer представя клиент на заведението.

Всеки клиент съдържа:

- име
- стратегия за отстъпка

Клиентът може да бъде:

- нов
- редовен
- VIP

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Project49.Strategy_Pattern;

namespace Project49.Customer
{
    4 references
    public class Customer
    {
        2 references
        public string Name { get; set; }
        3 references
        public IDiscountStrategy DiscountStrategy { get; set; }

        2 references
        public double ApplyDiscount(double price)
        {
            return DiscountStrategy.ApplyDiscount(price);
        }
    }
}
```

5.6 Клас Customer

Класът `Customer` представя клиент на заведението.

Всеки клиент съдържа:

- име
- стратегия за отстъпка

Клиентът може да бъде:

- нов
- редовен
- VIP

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Project49.Strategy_Pattern;

namespace Project49.Customer
{
    4 references
    public class Customer
    {
        2 references
        public string Name { get; set; }
        3 references
        public IDiscountStrategy DiscountStrategy { get; set; }

        2 references
        public double ApplyDiscount(double price)
        {
            return DiscountStrategy.ApplyDiscount(price);
        }
    }
}
```

5.7 Интерфейс `IDiscountStrategy`

Интерфейсът `IDiscountStrategy` реализира `Strategy Pattern`.

Той дефинира метод за:

- изчисляване на отстъпка

Чрез този подход могат лесно да се добавят нови типове отстъпки.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Project49.Strategy_Pattern
{
    2 references
    public interface IDiscountStrategy
    {
        2 references
        double ApplyDiscount(double price);
    }
}
```

5.8 Клас RegularCustomerDiscount

Класът RegularCustomerDiscount реализира интерфейса IDiscountStrategy.

Той:

- прилага отстъпка за редовни клиенти
- намалява крайната цена на поръчката

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Project49.Strategy_Pattern
{
    2 references
    public class RegularCustomerDiscount : IDiscountStrategy
    {
        2 references
        public double ApplyDiscount(double price)
        {
            return price * 0.9;
        }
    }
}

```

5.9 Клас Order

Класът Order отговаря за:

- съхранение на поръчаните ястия
- изчисляване на крайната сума

Съдържа списък с всички ястия, добавени към поръчката.

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Project49.Order
{
    4 references
    public class Order
    {
        public List<Dish> Dishes = new List<Dish>();

        2 references
        public double GetTotal()
        {
            return Dishes.Sum(d => d.Price);
        }
    }
}

```

5.10 Клас ReceiptBuilder

Класът ReceiptBuilder реализира Builder Pattern.

Той изгражда касов бон стъпка по стъпка:

- хедър
- продукти
- крайна сума
- информация за плащане

Този подход прави генерирането на касовия бон по-гъвкаво и лесно за поддръжка.

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Project49.Builder_Pattern
{
    4 references
    public class ReceiptBuilder
    {
        private string receipt = "";

        2 references
        public void AddHeader()
        {
            receipt += "FAST FOOD\n";
        }

        2 references
        public void AddItem(string name, double price)
        {
            receipt += $"{name} - {price} лв\n";
        }

        2 references
        public void AddTotal(double total)
        {
            receipt += $"Общо: {total} лв\n";
        }

        2 references
        public string GetReceipt()
        {
            return receipt;
        }
    }
}

```

5.11 Клас StatisticsManager

Класът StatisticsManager реализира Singleton Pattern.

Той отговаря за:

- съхраняване на статистика
- общ приход
- брой поръчки

Използването на Singleton гарантира, че в системата съществува само една инстанция на този клас.

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Project49.Singleton
{
    4 references
    public class StatisticsManager
    {
        private static StatisticsManager instance;

        1 reference
        private StatisticsManager() { }

        0 references
        public static StatisticsManager Instance
        {
            0 references
            get
            {
                if (instance == null)
                {
                    instance = new StatisticsManager();
                }
                return instance;
            }
        }

        0 references
        public double TotalRevenue { get; set; }
    }
}

```

6. Реализация на системата

Основната логика на приложението се намира в класа Program.

При стартиране на приложението:

1. Се създава ястие чрез Factory Pattern
2. Добавя се допълнителна съставка чрез Decorator Pattern
3. Създава се клиент
4. Прилага се отстъпка чрез Strategy Pattern
5. Генерира се касов бон чрез Builder Pattern
6. Обновява се статистиката чрез Singleton Pattern

```
using Project49.Builder_Pattern;
using Project49.Customer;
using Project49.Decorator_Pattern;
using Project49.Factory_Pattern;
using Project49.Order;
using Project49.Strategy_Pattern;
using Project49;

1 reference
class Program
{
    0 references
    static void Main()
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;

        Dish burger = DishFactory.CreateDish("burger");
        burger = new CheeseDecorator(burger);

        Customer customer = new Customer
        {
            Name = "Иван",
            DiscountStrategy = new RegularCustomerDiscount()
        };

        Order order = new Order();
        order.Dishes.Add(burger);

        double total = order.GetTotal();
        total = customer.ApplyDiscount(total);

        ReceiptBuilder builder = new ReceiptBuilder();
        builder.AddHeader();
        builder.AddItem(burger.Name, burger.Price);
        builder.AddTotal(total);

        Console.WriteLine(builder.GetReceipt());
    }
}
```

7. Тестване на приложението

Приложението беше тествано с различни сценарии:

- създаване на поръчка
- добавяне на съставки
- изчисляване на отстъпка
- генериране на касов бон

Всички функционалности работят успешно без runtime грешки.

8. Заключение

Разработеното приложение успешно реализира основните изисквания на проекта за заведение за бързо хранене.

Използването на шаблони за софтуерен дизайн подобрява:

- организацията на кода
- разширяемостта
- поддръжката на системата

Проектът демонстрира практическо приложение на:

- Factory Pattern
- Decorator Pattern
- Strategy Pattern
- Singleton Pattern
- Builder Pattern

Системата може лесно да бъде разширена с:

- графичен интерфейс
- база данни
- онлайн поръчки
- потребителски профили

9. Използвана литература

1. Gamma, E., Helm, R., Johnson, R., Vlissides, J. – Design Patterns: Elements of Reusable Object-Oriented Software

2. Microsoft Learn – C# Documentation
3. .NET Documentation
4. Software Engineering – Ian Sommerville
5. Object-Oriented Programming Concepts and Design Patterns

Линк: <https://github.com/DanjiAngelov/DesignPatterns-Factory>